

Lab 1 — Code Hint Sheet

Your First Network: the PyTorch training loop

Stuck on a **YOUR TURN** cell? Each line you need is below with a few options. Pick one, type it in, and **run the cell** — the built-in `assert` checks will tell you whether it is right. Only the two marked cells need editing; run everything else as-is.

Step 4 — the training loop. Write these five lines *in order* inside the `for epoch` loop (Lecture 2: forward $\rightarrow$ loss $\rightarrow$ zero $\rightarrow$ backward $\rightarrow$ step).

1. Forward — make a guess.

- (a) `pred = model(X_train)`
- (b) `pred = loss_fn(X_train)`
- (c) `pred = model.guess(y_train)`

2. Loss — how wrong?

- (a) `loss = loss_fn(pred, y_train)`
- (b) `loss = model(pred, y_train)`
- (c) `loss = loss_fn(X_train, y_train)`

3. Zero — clear last step's gradients.

- (a) `optimizer.zero_grad()`
- (b) `optimizer.reset()`
- (c) `model.zero_grad(loss)`

4. Backward — backprop the blame.

- (a) `loss.backward()`
- (b) `optimizer.backward()`
- (c) `model.backward(loss)`

5. Step — nudge every weight.

- (a) `optimizer.step()`
- (b) `optimizer.update()`
- (c) `loss.step()`

Step 6 — the learning rate: find all three regimes. You are given `lr_ok = 0.01`. Pick one rate that is *too small* (the loss barely moves in 150 epochs) and one that is *too big* (the loss climbs). The plot shows all three curves.

1. A learning rate that is TOO SMALL.

- (a) `lr_too_small = 0.0001` (100 $\times$ smaller: loss 0.277 $\rightarrow$ 0.213, still crawling)
- (b) `lr_too_small = 0.003` (smaller, but still trains well: ends at 0.017 — not a crawl)
- (c) `lr_too_small = 0.1` (that is *bigger* than `lr_ok`)

2. A learning rate that is TOO BIG.

- (a) `lr_too_big = 5.0` (huge steps overshoot: loss 0.277 $\rightarrow$ 0.487, it rose)
- (b) `lr_too_big = 0.1` (bigger, but it wobbles and still ends lower, at 0.073)
- (c) `lr_too_big = 0.001` (that is *smaller* than `lr_ok`)

Then explore: the next cell has no blank. Change `lr_try`, re-run it, and each run is added to the plot. Try 0.001, 0.003, 0.03, 0.1, 0.3, 1.0 and compare the final losses.

Step 7 — evaluate on the test set. Predict without learning, threshold at 0.5, then measure the fraction correct.

1. Predict without gradients.

- (a) with `torch.no_grad():` (then, indented) `probs = model(X_test)`
- (b) with `torch.grad():` (then, indented) `probs = model(X_test)`
- (c) `probs = model.no_grad(X_test)`

2. Turn probabilities into 0/1 predictions.

- (a) `predicted = (probs > 0.5).float()`
- (b) `predicted = probs.round(threshold=0.5)`
- (c) `predicted = probs.argmax()`

3. Accuracy — fraction that match the label.

- (a) `test_acc = (predicted == y_test).float().mean().item()`
- (b) `test_acc = (predicted - y_test).mean()`
- (c) `test_acc = accuracy(predicted, y_test)`

Step 8 — did it overfit? compute the training accuracy the same way. Mirror the Step 7 evaluation, but on `X_train/y_train`, then compare to `test_acc`.

1. Training accuracy (same recipe as Step 7, on the training data).

- (a) with `torch.no_grad()`: `probs_train = model(X_train)` then
 `predicted_train = (probs_train > 0.5).float()`
 `train_acc = (predicted_train == y_train).float().mean().item()`
- (b) `train_acc = (model(X_test) == y_test).float().mean()` (that's the *test* set, not train)
- (c) `train_acc = loss_history[-1]` (that's a loss, not an accuracy)

2. Overfitting means the model scores HIGHER on data it has seen. Fill the comparison.

- (a) `overfitting = train_acc > test_acc` (train beats test $\Rightarrow$ overfit)
- (b) `overfitting = train_acc < test_acc` (backwards — and the assert will fail)
- (c) `overfitting = train_acc == test_acc` (they're not equal here)

*Answers self-check in the notebook: if the **assert** cells print their success message, you chose right. The loss should fall steeply and then flatten; the too-big learning rate makes the loss rise instead; test accuracy should land around **0.72–0.77** (coin-flip baseline is 0.50); and training accuracy should be clearly higher than test accuracy (a train/test gap of roughly **0.20–0.25**) — that gap is overfitting.*